LexLean closes a sentence. PrismPM closes a system. The second mirror takes one language-1.1 snapshot and reflects it into architecture, package, guest, view, and archive, under one identity. A facet that cannot be regenerated from that snapshot is not a Foundry system.

## What the compiler is

PrismPM is the platform-model compiler for the workshop. A model is a closed `.lex.tex` source graph, authored in LexLean language 1.1. The normative contract is `SPEC.md`. Version 0.1.0 was a systems-modeling prototype. Version 0.2.0 completed the portable-application line. Version 0.3.0 is the accepted production-system SDK: it adds lifecycle and system profiles while preserving every 0.1.x application API and the Calculator artifact contract.

The prohibition is the joint. Every Prism-owned formal type, value, validator, proposition, theorem, and proof is authored in `.lex.tex` and generated by LexLean. Handwritten Prism `.lean`, `lakefile.lean`, Lean wrappers, exporter modules, tactics, raw Lean payloads, and selectable compiler backends are forbidden. Generated Lean is a build artifact or a manifest-owned reviewed golden, never authoritative source. `prism-stdlib` itself is authored strictly in `.lex.tex` (`ST-05`). Application behavior is never supplied by target-specific application code. `lean4-prod` generates the Cargo core, the Core-Wasm guest, and the View adapters without application-specific branches.

Configuration cannot smuggle a second path. `prismpm.toml` has a closed shape — `spec = "prismpm/project/1"`, project name, `lexlean_project`, `build_root`, and three limits — and unknown fields are errors. It cannot select language backends, verifier stages, toolchains, standards, runtime roots, or dependency revisions. Those are closed by `prismpm.lock` and `standards.lock`. The 0.3.0 interface is a digest-pinned Linux SDK OCI index for `linux/amd64` and `linux/arm64`. Inside it, Rust 1.97.1 and Lean 4.32.1 are mandatory.

## Five stages, three honesty levels

```text
```

```
.lex.tex (language 1.1) + lock
        |
        v
1  LexLean snapshot, Lean, LaTeX, maps, coverage
        |   observed axioms == declared policy
        v
2  lean4-prod: named roots only, complete LCNF
        |
        v
3  generic generators: Cargo core, core-wasm guest, View adapters
        |
        v
4  prism-stdlib Holo/1 projection: capability, manifest, directory,
        provenance, portable View, source manifest
        |
        v
5  Hologram v4 fat archive, then pinned Live oracle
```

`check`, `build`, and `verify` are not synonyms.

`check` loads LexLean, obtains `lexlean/semantic-snapshot/1`, and projects the canonical Prism model document in memory against the generated stdlib contract. It enforces limits and writes nothing. It does not launch Lean, Lake, `leanchecker`, `lean4-prod`, rustc, or a generated executable.

`build` invokes LexLean's deterministic build and publishes under `.prism/build/<prism-build-id>/`. Publication takes an exclusive lock, writes a unique staging directory, and atomically renames it. Extra files, symlinks, missing files, and tampering are errors. Build makes no verification claim.

Only `verify` may emit `prismpm/application-acceptance/1`. It reruns and binds generated-package, Wasm, View, archive, oracle, and modeled-vector checks, including the LexLean declaration audit: imported namespaces, exact source axiom policies, empty theorem axioms, elaboration, and kernel replay.

That split is the same honesty rule as LexLean, one layer up. A workshop may hold a model. It may not call the model accepted until `verify` has bound the reflections.

## Viewpoints are lexicon packages, not diagrams

PrismPM validates a repository-authored formal profile across viewpoints. It does not reproduce a standard, grant certification, or prove facts a standard merely asserts. ISO/IEC

25019, the other parts of ISO/IEC 27034, quality-in-use outcomes, and organizational certification sit outside the Holo/1 baseline.

What is inside is three language-1.1 packages, exact local locks, builtin packages only:

| Facet | Package | Standard surface |
|---|---|---|
| `prism-arch` | `prism.arch@1.0.0` | ISO/IEC/IEEE 42010:2022 architecture description (`FT-01`) |
| `prism-sec` | `prism.sec@1.0.0` | ISO/IEC 27034-1:2011 and 27034-5:2017 application security; ISO/IEC 27005:2022 risk (`FT-02`, `FT-04`) |
| `prism-qual` | `prism.qual@1.0.0` | ISO/IEC 25010:2023 product quality (`FT / ST-04`) |

`prism-stdlib` defines the corresponding primitives in `Foundation.Arch`, `Foundation.Sec`, and the quality module, and exports registered runtime validator roots (`ST-08`). A Foundry site, role, or control is therefore not “aligned with 42010” because a document says so. It is aligned only if it is an entity in `prism.arch` that the projector can index and the validator can refuse.

Conformance families that police this mirror include `RP` (repository), `FT` (facets), `ST` (stdlib), `HO` (Holo), `CT` (controller), `AR` (artifacts), `EX` (execution), `VR` (verification), `SE` (security boundary), `AU` (authority), `SY` (system), `DK` (one SDK inventory), `SC` (supply chain), and `LC / DP / OP` (lifecycle, deployment, operations). `DK-01` is the inventory claim: one versioned SDK closes over PrismPM, LexLean, Lean, lean4-prod, stdlib, oracles, adapters, and schemas.

## Identity is the mirror’s invariant

The Prism build ID is SHA-256 of canonical build-input bytes: LexLean source, semantic, compiler, snapshot, and build IDs; stdlib semantics and package identity; lean4-prod and Hologram pins; target profile; generated core, guest, View, browser, manifest, capability, extension, and Hologram application identities; dependency-register digest. Non-application builds use `prismpm/build-inputs/1`. Application builds use `prismpm/build-inputs/2`, which adds `application_artifacts_sha256` over the path-sorted manifest rows. The self-containing `manifest.json` is excluded, so source-free release verification can recompute the binding from checked artifact rows. Legacy application `/1` inputs lack that binding and fail with `PP6101`. They are rebuilt and reverified, not relabeled.

Two documents must not be confused:

- `model.prism.json` is the canonical Prism model document, a total deterministic function of a valid LexLean snapshot ( `HO-04` ). Entity identifiers are qualified strings with deterministic zero-based indexes ( `HO-05` ). Identity is SHA-256. JSON is never a `.holo` .
- Holo/1 is the application profile defined by generated declarations under `Foundation.Holo.V1` . It is not a physical version. Its container is a Hologram archive, physical version 4, header `HOLO` plus little-endian version 4 and canonical flags ( `HO-01` ). The wire codec is LexLean-generated `Foundation.Holo.V1.Wire` in `prism-stdlib` . Hologram platform code is not a production Cargo dependency; the Live oracle is pinned.

The producer extension key is `https://uor.foundation/extension/prismpm-model/v1` . Its `prismpm/model-provenance/1` payload binds the model blob, LexLean and lean4-prod evidence, generated core/package/guest/View/browser evidence, dependency commits, and application kappa. It cannot grant a capability or replace the manifest. Archive fingerprint, archive kappa, and Prism attestation are bound after construction, so the digest does not cycle.

A strict Calculator fat archive has one manifest, one Metadata section, two extensions (upstream application directory and Prism provenance), four content blobs, and one footer. Unknown, duplicate, missing, reordered, overlapping, truncated, trailing, over-limit, or cross-reference-invalid content is rejected. Platform-independent artifacts exclude host paths, timestamps, and unstable environment data ( `HO-06` , `AR-04` ).

## What the generators are allowed to emit

From the named verified roots, generic generators produce:

- a Cargo crate and registry package — after publication, an ordinary Rust consumer depends on the generated API, as in `prism-calculator = "=0.1.0"` and `calculate(Operation::Add, 20, 22) == Ok(42)` ;
- an import-free `hologram:guest/core-wasm@1` guest;
- an evaluated View, lowered to portable HOLOVIEW plus a browser adapter and assets;
- the binary `ApplicationName.holo` .

Execution constraints are part of the mirror, not a runtime preference. Generated Rust validators are `no_std` and do no heap allocation during validator calls ( `EX-03` ). The host decodes canonical Holo and normalizes string IDs to indexes ( `EX-04` ). `lean4-prod` must export compilable LCNF and close the transitive dependency set without opaque gaps ( `EX-01` , `EX-02` ). Browser-resident system releases bind selected proved applications, mandatory controls, exact generated artifacts, locked projection oracles, and source-free replay ( `SY-08` ). Caller browser-cache overrides cannot select another binary.

The independent oracle record is `prismpm/hologram-oracle/2`. The prior recording-only `/1` report cannot satisfy current application verification. Calculator's oracle acceptance covers 8 legacy numeric cases and 10 UTF-8 text cases.

## Calculator is the closure witness

v0.3.0 is releasable only when all of these pass the atomic release contract together: generated `prism-stdlib`, generated `prism-calculator`, `Calculator.holo`, independent Hologram execution evidence, the public `calculator-example` Pages application, and the complete CalculatorSystem reference. Release status records that closure across six release steps.

`calculator-example` does not reimplement the calculator. It mirrors the exact content-addressed model, reruns PrismPM acceptance, imports the registry crate, and deploys only the generated six-file browser closure. The reference closure runs that same SDK and system reference across Compose, Kubernetes, and Pages, with distinct Release A/B digests. Kubernetes adapters are fail-closed, read-only roots, security profiles, two-stage deployment.

That is why Foundry consumes PrismPM as an SDK on the Calculator pattern. `uor-foundry` owns organization lifecycle, workflows, scoped authority, and portal Views. `foundry-web` publishes the exact verified portal and checks the live deployment. Neither is allowed a second implementation of the platform. A Foundry increment that has not passed the same shape of closure — generated stdlib profile, archive, oracle evidence, portal handoff — remains a draft, the way `src/Foundry.lex.tex` is still an unaccepted local preview.

## Why this is the second mirror

The first mirror made prose and proof two faces of one sentence. The second makes architecture, security, quality, package, guest, view, and archive faces of one system. Identical identity is the contract: build ID, model SHA-256, application kappa, and provenance payload either agree, or the artifact is rejected.

Phase-mirror methodology at this layer is not "more views of the organization." It is one civic model, reflected only through generic generators, admitted only by `verify`, and sealed so a foreign oracle can execute the reflection without trusting the workshop that authored it. A handwritten path would be an expansive term: behavior the snapshot cannot regenerate. That term does not enter the base-state.